This skill should be considered when you need to enhance a given infeasible or non-optimal fjsp baseline to a feasible schedule with less makespan considering downtime constraints, job precedence violations, policy budget constraints. The following constraints should be satisfied. end(j, o) <= start(j, o+1). Here j means the job index and o means the operation index. There should be no overlaps on the same machine or with downtime windows. Keep the number of machine changes and the total L1 start-time shift within the policy budgets. You can calculate the number of machine changes by MC = \sum_{(j,o)} [m_{new}(j,o) \ne m_{base}(j,o)]. You can calculate the total L1 start-time shift by Shift_{L1} = \sum_{(j,o)} |start_{new}(j,o) - start_{base}(j,o)|.
To achieve this, never start any operation earlier than the baseline. When repairing operations in precedence-aware order, place each operation at the earliest feasible time. Anchor is calculated by anchor(j,o) = \max(start_{base}(j,o), end_{new}(j,o-1)), end_{new}(j,o-1) is end time of the previous operation of the same job at the new schedule. If start > anchor, then start-1 must be infeasible. You guarantee this by scanning integer time forward by +1 from anchor. Jumping to “next gap” without checking every integer may break minimality. If the given baseline is invalid, replace the machine with a feasible one.

Here is the pipeline. For each operation, first find the earliest time to start. It cannot start earlier than the baseline, and it cannot start before the previous operation of the same job finishes. Then list only the machines that are allowed for this operation, and use the processing time that belongs to each machine. For each candidate machine, find the allowed earliest time and pick the first start time that does not overlap with other work on that machine and does not fall into any downtime window. Choose the option that makes the smallest changes overall. Prefer not changing machines and keeping start-time shifts small, and make sure you stay within the policy budgets. After selecting a start time, immediately record this operation on the machine timeline in the same precedence-aware order, so the result matches the evaluator’s step-by-step simulation.

Here are reference codes.
```python
# sorted Downtime windows
downtime[m] = sorted([(start,end), ...])
def overlap(s,e,a,b):
    return s < b and a < e
```

```python
# Precedence-aware repair order
def precedence_aware_order(base_list):
    base_map = {(r["job"], r["op"]): r for r in base_list}
    base_index = {(r["job"], r["op"]): i for i, r in enumerate(base_list)}
    keys = list(base_map.keys())
    keys.sort(key=lambda k: (k[1], base_map[k]["start"], base_index[k]))
    return keys
```

```python
def earliest_feasible_time(m, anchor, dur, machine_intervals, downtime, safety=200000):
    t = int(anchor)
    for _ in range(safety):
        if not has_conflict(m, t, t+dur, machine_intervals, downtime):
            return t
        t += 1
    return t
```

```python
def has_conflict(m, st, en, machine_intervals, downtime):
    for a,b in machine_intervals.get(m, []):
```

```
        if overlap(st,en,a,b):
            return True
    for a,b in downtime.get(m, []):
        if overlap(st,en,a,b):
            return True
    return False
```

```python
# Baseline machine may be illegal
base_m = base_map[(j,o)]["machine"]
if base_m not in allowed[(j,o)]:
    # baseline is invalid; pick a legal default (min duration is a good heuristic)
    base_m = min(allowed[(j,o)], key=lambda m: allowed[(j,o)][m])
base_d = allowed[(j,o)][base_m]
```

```python
#Use a lexicographic score that matches your priorities
machine_change = int(mm != base_m_orig)
start_shift    = abs(st - base_start)
score = (machine_change, start_shift, st, mm)
#Then pick the smallest score, but respect remaining machine-change budget.
```

```python
#A naive “always keep baseline machine” can cause large start shifts. This often reduces `Shift
_L1` enough to pass tight budgets without exploding machine changes. Use a simple trigger to co
nsider alternates *only when it helps*:

THRESH = 6  # tune; small instances often 3~10 works
# First try baseline machine
cand = best_candidate_restricted_to([base_m])

# If shift is large and we still can change machines, search alternates
if (cand.start - base_start) >= THRESH and mc_used < max_mc:
    cand2 = best_candidate_over_all_allowed_machines()
    if cand2.start < cand.start:   # or cand2.shift < cand.shift
        cand = cand2
```